MacZine
THE MACTECH SOLUTIONS NEWSLETTER
ISSUE Nº 028 · AUGUST 2026 · MACTECHSOLUTIONSLLC.COM/MACZINE

FIELD REPORT · CONFIGURATION MANAGEMENT

STIG vs CIS Benchmark: Which Baseline Counts as Evidence

DISA STIGs and CIS Benchmarks compared: who publishes each, who is bound by which, how their severity ratings differ, and what an assessor accepts.

BY **MAXINE** READING TIME · 8 MIN STIG · CIS · CONFIGURATION-MANAGEMENT · NIST-800-171

Ask a program manager how their servers are configured and the honest answer, more often than not, is "to best practices" - a phrase that satisfies nobody, least of all the assessor sitting across the table from 3.4.1 and 3.4.2. Both are 5-point requirements in the DoD Assessment Methodology, among the heaviest weights any single control carries, and both get hand-waved with a confidence that has nothing behind it. A baseline is not a philosophy. It is a specific, documented, enforced configuration for each type of system you run, with every departure from it written down and a way of noticing when reality drifts away from the document. Here is the sequence that gets you there, in the order an assessor actually walks it.

FIELD COPY
ISSUE Nº 028
STIG

What actually qualifies as a baseline

Start with the question readers actually have, because it is simpler than the anxiety around it suggests: a DISA Security Technical Implementation Guide, a CIS Benchmark, or your own documented build standard all qualify. None of the three is mandated for a contractor's own network. STIGs bind systems operating under an Authorization to Operate through the Risk Management Framework; a contractor's internal boundary is scoped differently, and nothing in NIST SP 800-171 names DISA's catalog by number. What 3.4.1 and 3.4.2 require is that you pick one, document it per system type, and apply it - a RHEL 9 database server and a Windows 11 workstation each get their own defined configuration, not one aspirational paragraph covering both.

An assessor will accept any of the three sources, and picking one is mostly a question of what you already run. A shop standardized on Red Hat and Windows Server gains real leverage from adopting DISA's STIGs outright, because the check and fix text already exists at the level of individual registry keys and configuration lines. CIS Benchmarks cover a wider vendor list and read a shade less prescriptive. A homegrown build standard is legitimate too, provided it specifies settings at the same granularity the other two do - "harden the OS" is not a setting, and neither is a screenshot of a checklist nobody re-ran since the box was imaged.

Where a STIG and a CIS Benchmark actually differ

Since the two names get used interchangeably and are not interchangeable, it is worth being precise about what each one is. A STIG is published by the Defense Information Systems Agency, and it exists to harden systems the Department of War operates or authorizes. It is written as an order rather than a recommendation, its content descends to individual registry keys and configuration lines, and every finding carries a severity category from CAT I down to CAT III. A CIS Benchmark comes from the Center for Internet Security, a nonprofit, and is built by a consensus process across vendors and practitioners rather than issued by an authority. It covers a far wider range of platforms, cloud services and containers among them, and it ships in profile levels: Level 1 for settings that harden a system without disturbing how it is used, Level 2 for defence in depth on systems where some loss of functionality is an acceptable trade.

The practical consequence is that a STIG is usually the stricter document and the CIS Benchmark the broader one, and that neither binds a defense contractor's internal network by name. The useful middle path is not widely known:

5

POINTS EACH 3.4.1 AND 3.4.2 CARRIES IN THE DOD ASSESSMENT METHODOLOGY - AMONG THE HEAVIEST WEIGHTS ANY SINGLE REQUIREMENT GETS

2,029

DISA STIG RULES IN MACTECH'S REFERENCE SERVICE, ACROSS 15 BENCHMARKS COVERING RHEL, UBUNTU, WINDOWS, WINDOWS SERVER AND CISCO IOS, NX-OS AND ISE

4

ARTIFACTS AN ASSESSOR ASKS FOR TO EVIDENCE A BASELINE - MOST PROGRAMS WALK IN WITH ONE

CIS publishes STIG-aligned profiles for several of its benchmarks, which lets a shop already standardized on CIS content produce something an assessor familiar with DISA's vocabulary reads without translation. If you are running systems under an ATO as well, take the STIG and get one artifact for both programs. If your estate is mixed and includes platforms DISA never wrote a STIG for, start from CIS and document where you departed from it.

What an assessor will not accept

Two things fail regardless of which source you picked. The first is a vendor default left unmodified - not because a default is automatically insecure, but because 3.4.2 asks you to enforce settings you chose, and a setting nobody touched was not chosen. The second, more common failure is a baseline that exists in a Word document and nowhere else: a hardening guide written once during procurement, filed, and never run against the machines it describes. An assessor's method is to sample - pull applied configuration off a live host and compare it to the document - and a baseline that only exists on paper fails the moment that sample is taken. The document has to be a description of what the machines actually are, not what someone once intended them to be.

Build the deviation register before the assessor finds it for you

No real environment holds a benchmark line for line. An application needs a service the STIG disables by default; a monitoring agent needs a port the CIS Benchmark closes; a legacy client can't negotiate the cipher suite the hardening guide mandates. That gap is not the failure. Every mature program has one somewhere. What separates a mature program from a failed one is whether the gap is written down with a rationale and a compensating control, or whether the assessor finds it first by running a scan you didn't run yourself.

A deviation register does three things for every departure: names the specific rule or setting not being enforced, states why - the business or technical reason the strict setting can't hold - and records what stands in its place, whether that's a compensating control, a network-layer restriction, or an accepted risk signed by someone with the authority to accept it. A register with zero entries is not a sign of a perfectly held baseline. In an environment running real applications, it is usually a sign nobody has looked closely enough to find the gaps that are already there.

A deviation nobody wrote down isn't a secret you kept. It's a finding you handed the assessor for free.

Baselines rot: catch drift with 3.4.3 and 3.11.2

Baselines pass on day one and rot quietly afterward, and that decay - not the initial build - is the actual failure mode auditors see most. A patch changes a default. An administrator opens a port under deadline pressure and forgets to close it. A new server gets imaged from a golden template that was itself never updated after the last STIG release. None of this shows up unless something is watching for it, which is why 3.4.1 and 3.4.2 don't stand alone - they lean on 3.4.3, change control, and 3.11.2, vulnerability scanning, as the mechanisms that notice.

"Regularly" is not a cadence an assessor will credit. A working program runs credentialed vulnerability scans on a fixed schedule - monthly is the common floor for a CUI boundary - and re-checks applied configuration against the baseline on its own cycle, quarterly at minimum and tighter around any DISA STIG or CIS Benchmark release. Every scan finding that traces back to a configuration setting, not a missing patch, either gets remediated on a tracked timeline or becomes a new line in the deviation register. Change control closes the loop: nothing that alters a baselined setting ships without a ticket, and the ticket is the record that connects the drift back to a decision someone made on purpose.

The four artifacts, and why most programs have one

Strip the whole program down to what an assessor actually asks for and it is four things: the baseline document itself, specifying settings per system type; applied-configuration output pulled from a representative sample of hosts, proving the document and the machines agree; the deviation register, with a rationale and a compensating control behind every entry; and the change tickets that authorized each deviation and each subsequent modification. Most programs that fail this control on assessment day have written the first artifact and stopped. The other three are where the actual evidence lives, and they're also where a MacTech readiness assessment spends most of its time, because a gap in any one of the four reads to an assessor exactly like a gap in the whole control.

◆ CAT I, II AND III

STIG findings carry a severity category. **CAT I** is a weakness that would directly and immediately result in a loss of confidentiality, availability or integrity. **CAT II** degrades protection and may lead to one. **CAT III** reduces defence in depth without creating an exposure on its own. Assessors read the category before they read the count, which is why a hundred open CAT IIIs alarms a program less than one open CAT I.

◆ WHAT A VENDOR DEFAULT ACTUALLY FAILS

An unmodified factory image is not the absence of a baseline - it is someone else's baseline, tuned for compatibility across every customer that vendor has, not for a CUI boundary. 3.4.2 asks you to enforce settings you chose. A default you never touched fails that test even when it happens to be reasonably secure.

Running RMF and CMMC evidence off the same checklist

For a contractor running both an ATO under RMF and a CMMC program, the STIG format does double duty. DISA's .ckl checklist is the standard artifact for documenting compliance against a specific STIG, rule by rule, and it is the format a Security Control Assessor expects to see under RMF. The general case for treating RMF and CMMC as one evidence problem instead of two separate ones was made here in Issue Nº 012; configuration baselines are one of the cleanest places to apply it, because the same .ckl output that proves a system to a Security Control Assessor is also the applied-configuration artifact a C3PAO wants for 3.4.1 and 3.4.2. MacTech's own STIG reference service tracks 2,029 rules across RHEL 8, RHEL 9, Windows 11, Windows Server 2022, and Cisco IOS Router NDM, with per-rule check and fix text, NIST control mappings, and export to that same .ckl format - one baseline, evidenced once, read by both programs.

The document you already wrote is not the finish line. It's the first of four artifacts, and the only one that doesn't change once the ink dries. Programs that treat it as the whole answer are the ones an assessor spends the least time being polite to. Talk to a director if your baseline evidence currently lives in one file instead of four. ◆